Reviewer Pack — Minimal Representation Rank and Frege Proof Width Correlatio...

Entry bd993bcd755e · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-04 23:46:35 UTC

Minimal Representation Rank and Frege Proof Width Correlation

Entry ID: `bd993bcd755e` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-04 23:46:35 UTC

1. Conjecture

Field A (mathematical branch): Representation Theory (Group Representations) **Field B** (complexity object): Resolution Proofs (Frege Proof Complexity)

Statement:

> For every group G, the minimal representation rank (mrank(G)) of its irreducible representations is linearly correlated with the Frege proof width of its associated Tseitin formula φ_G, such that mrank(G) = Θ(w(φ_G)).

Rationale (proposer's reasoning):

> The study of minimal representation ranks in group theory could reveal nontrivial properties of complex structures. These properties might translate into bounds on proof complexity, specifically the Frege proof width for Tseitin formulas derived from groups, potentially leading to new insights in computational complexity.

Taxonomy category: `RepresentationTheory` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `e45250ee375cdc7b`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> We will consider the conjecture supported if for all 30 random group seeds, the Pearson correlation coefficient r between minimal representation rank (mrank(G)) and Frege proof width (w(φ_G)) is greater than or equal to 0.7, with p-value ≤ 0.01, and no seed produces a metric value > 10 for either mrank(G) or w(φ_G).

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	SAFE	SAFE
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_group(n):
        # Generate a random group G with n elements using Cayley's theorem
        generators = [random.sample(range(1, n), 2) for _ in range(random.randint(1, 3))]
        relations = []
        for gen in generators:
            rel = [(gen[0], gen[1]), (gen[1], gen[0])]
            relations.extend(rel)
        return generators, relations

    def minimal_representation_rank(generators, relations):
        # Compute the minimal representation rank of the group G
        n = len(generators)
        mrank = 0
        for rel in relations:
            if rel not in generators and (rel[1], rel[0]) not in generators:
                mrank += 1
        return mrank

    def tseitin_formula(n):
        # Construct the Tseitin formula φ_G for the group G
        clauses = []
        for i in range(1, n+1):
            for j in range(1, n+1):
                if i != j:
                    clause = [f'x{i}', f'x{j}']
                    clauses.append(clause)
        return clauses

    def frege_proof_width(clauses):
        # Compute the Frege proof width of the Tseitin formula φ_G
```

```
    width = 0
    for clause in clauses:
        width = max(width, len(clause))
    return width

n_values = [5, 10, 15, 20, 30, 40]
results = []

for n in n_values:
    generators, relations = generate_group(n)
    mrank = minimal_representation_rank(generators, relations)
    clauses = tseitin_formula(n)
    w_phi_G = frege_proof_width(clauses)

    if mrank is None or w_phi_G is None:
        return {
            "metric_name": "mrank(G) vs. w(φ_G)",
            "metric_value": None,
            "instances_tested": 1,
            "n_max": n,
            "conjecture_holds": False,
            "counterexample": "mapping_undefined"
        }

    results.append({
        "mrank": mrank,
        "w_phi_G": w_phi_G
    })

if len(results) < 30:
    return {
        "metric_name": "mrank(G) vs. w(φ_G)",
        "metric_value": None,
        "instances_tested": len(results),
        "n_max": max(n_values),
        "conjecture_holds": False,
        "counterexample": "not_enough_instances"
    }

mrank_values = [r['mrank'] for r in results]
w_phi_G_values = [r['w_phi_G'] for r in results]

mean_mrank = sum(mrank_values) / len(mrank_values)
mean_w_phi_G = sum(w_phi_G_values) / len(w_phi_G_values)

if any(val > 10 for val in mrank_values + w_phi_G_values):
    return {
        "metric_name": "mrank(G) vs. w(φ_G)",
        "metric_value": None,
        "instances_tested": len(results),
        "n_max": max(n_values),
        "conjecture_holds": False,
        "counterexample": "value_exceeds_10"
    }

return {
```

```
        "metric_name": "mrank(G) vs. w(φ_G)",
        "metric_value": mean_mrank * mean_w_phi_G,  # Using product as a proxy for correlation
        "instances_tested": len(results),
        "n_max": max(n_values),
        "conjecture_holds": False,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43,

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    if all(r['conjecture_holds'] for r in results):
        mean_metric_value = sum(r['metric_value'] for r in results) / len(results)
        support_fraction = 1.0
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0 support_fraction={support_fractio
    else:
        first_failing_seed = next((r['seed'] for r in results if not r['conjecture_holds']), None)
        print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_f
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
mple': 'not_enough_instances'}
TRIAL: {'metric_name': 'mrank(G) vs. w(φ_G)', 'metric_value': None, 'instances_tested': 6, 'n_max': 4
TRIAL: {'metric_name': 'mrank(G) vs. w(φ_G)', 'metric_value': None, 'instances_tested': 6, 'n_max': 4
TRIAL: {'metric_name': 'mrank(G) vs. w(φ_G)', 'metric_value': None, 'instances_tested': 6, 'n_max': 4
TRIAL: {'metric_name': 'mrank(G) vs. w(φ_G)', 'metric_value': None, 'instances_tested': 6, 'n_max': 4
TRIAL: {'metric_name': 'mrank(G) vs. w(φ_G)', 'metric_value': None, 'instances_tested': 6, 'n_max': 4
TRIAL: {'metric_name': 'mrank(G) vs. w(φ_G)', 'metric_value': None, 'instances_tested': 6, 'n_max': 4
TRIAL: {'metric_name': 'mrank(G) vs. w(φ_G)', 'metric_value': None, 'instances_tested': 6, 'n_max': 4
TRIAL: {'metric_name': 'mrank(G) vs. w(φ_G)', 'metric_value': None, 'instances_tested': 6, 'n_max': 4

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_90793255.py", line 130, in <module>
    first_failing_seed = next((r['seed'] for r in results if not r['conjecture_holds']), None)
                              ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_90793255.py", line 130, in <genexpr>
    first_failing_seed = next((r['seed'] for r in results if not r['conjecture_holds']), None)
                                ~^^^^^^^^
KeyError: 'seed'
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test crashed before producing data, which means that the pre-registered support condition could not be unambiguously met. | next: Investigate and fix the crash in the test code to ensure it can complete without errors.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11707
2	propose	ollama_remote	glm4:latest	0	0	12854
3	preregistration	ollama_remote	glm4:latest	0	0	9772
4	novelty	ollama_remote	glm4:latest	0	0	10030
5	novelty	ollama_remote	glm4:latest	0	0	8783
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	41409
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8623
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9176
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14016
10	judge	ollama_remote	glm4:latest	0	0	20371

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 146742 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/bd993bcd755e.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/bd993bcd755e.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/bd993bcd755e.tar.gz` (if generated)